A Multi-Agent Academic Helpdesk for IIT Jodhpur: Retrieval-Augmented Question Answering and Safe Refusal

Autonomous Systems – Assignment 2 Report

Group: Akanksha Singh (B23CM1005), Shweta Mandal (B23CM1037)

Abstract

Students at an institute such as IIT Jodhpur routinely need answers that are scattered across Senate-approved regulations, programme curricula, academic calendars, timetables and faculty directories. This assignment implements an **Academic Helpdesk** that answers such queries from the institute's own documents, drafts emails to instructors, and declines questions that fall outside the academic domain. The system is organised as a multi-agent pipeline: a central **Orchestrator** classifies each query and forwards it to one of three specialised agents – a **PolicyAgent** (rules, regulations and curriculum, backed by retrieval over a ChromaDB vector store), a **ScheduleAgent** (slots, rooms and floated courses, backed by Pandas lookups over CSV tables) and an **EmailAgent** (formal leave and make-up examination emails). Six small tools connect the agents to data, and every refused query is appended to a JSONL log for human review.

The knowledge base consists of **32 official PDF documents** (regulations, Senate circulars, programme curricula and academic calendars) that were split into **1,075 chunks** and indexed with page-level metadata so that every retrieved passage can be cited. Four structured CSV files (timetable, floated courses, faculty, instructor mapping) hold the schedule data. A 20-prompt test suite covering policy, curriculum, timetable, course-catalogue, email and refusal queries is recorded as passing 20/20, with 100 % routing accuracy. An important property of the delivered code, which this report documents openly, is that the agents are **deterministic and rule-based**: the runtime path does not call an LLM, although the original plan specified Google ADK with Ollama-hosted models. Section 8 discusses what this implies and how the LLM layer could be added.

1 Objectives

- Build a helpdesk that answers rules, curriculum, timetable and course-catalogue questions using only institute documents and data.
- Return source citations (document name and page) for every answer produced from retrieval.
- Decompose the task into an orchestrator and specialised agents with clearly separated tools.
- Draft – but never send – formal emails to the correct instructor, resolved from the course code.
- Refuse non-academic questions with an explicit "I do not know" and log them for later review.
- Run entirely on a laptop with local resources (local persistent vector store, no cloud service at query time).
- Evaluate behaviour on a fixed prompt set for routing, keyword accuracy, citation presence and refusal.

2 Problem Statement

Academic information at the institute is fragmented across formats and owners. Attendance rules live in a 2024 Senate circular that supersedes part of the 2019 Academic Regulations; credit requirements are spread over a dozen programme-specific curriculum booklets; slot allocations are published as spreadsheets; and instructor contact details sit in separate directories. A student who wants to know, for example, whether low attendance still bars them from the Major examination must find the correct document, notice that an amendment supersedes the older text, and read it carefully.

The assignment therefore asks for an assistant that can handle the following seven query types, each of which is represented in the test suite:

#	Capability	Example prompt used in testing
1	Policy / regulation	What is the attendance requirement to sit in the Major exam?
2	Curriculum / credits	How many Open Elective credits do I need to earn in order to get a B.Tech. Degree?
3	Timetable	What is the slot for the course Autonomous Systems?
4	Course catalogue	How many 400 level electives have been floated by the Humanities department in this semester?
5	Email drafting	Write an email to the course instructor of CSL7570 that I need leave for 2 days
6	Out-of-scope refusal	What are the available dining options on the campus?
7	Unanswered-question logging	Every refused query is persisted with a timestamp and a reason.

The central difficulty is **trustworthiness**: a helpdesk that guesses is worse than none, because students act on what it says. The design therefore prioritises grounding (answers come from retrieved text or structured tables), citation, and a conservative refusal path over conversational fluency.

3 System Architecture and Workflow

3.1 Overview

Figure 1 shows the implemented architecture. The user interacts through a command-line loop (src/app.py). Each query is passed to AcademicHelpdeskOrchestrator.handle(), which returns a dictionary containing the answering agent, the detected intent and the answer text. The orchestrator owns three agent objects and one tool of its own (the unanswered-question logger); each agent in turn calls the tools it needs.

3.2 Agent and tool responsibilities

Component	Responsibility	Data / tools used
Orchestrator	Intent classification, routing, refusal of out-of-scope queries, logging	log_unanswered_question
PolicyAgent	Rules, regulations, curriculum and credit questions with citations	search_knowledge_base → ChromaDB (1,075 chunks)
ScheduleAgent	Slot / time / room lookup; counts and lists of floated courses	lookup_timetable, lookup_courses → timetable.csv, courses_floated.csv
EmailAgent	Extract course code, duration and email type; draft the message	get_instructor_email, draft_email → instructor_course_mapping.csv, faculty.csv

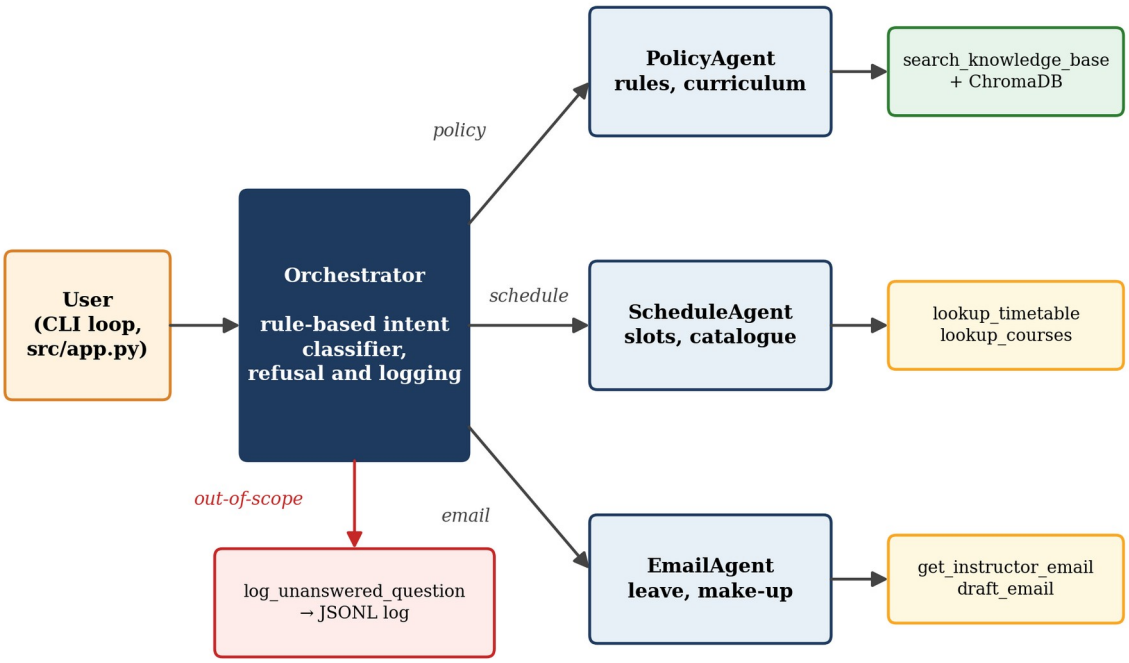


Figure 1: *Implemented system architecture (orchestrator, three specialised agents and six tools).*

3.3 Query workflow

1. The user types a query in the CLI loop.
2. classify_intent() lowercases the query and tests keyword groups in a fixed priority order (Figure 2).
3. For **out-of-scope** queries the orchestrator returns the refusal message, writes a log entry, and stops – no retrieval is attempted.
4. Otherwise the selected agent runs its tools: vector search for policy questions, DataFrame filtering for schedule questions, or mapping lookup plus template filling for emails.
5. The agent returns an answer (with a “Sources” block for retrieval answers); the CLI prints the agent name, intent and answer.

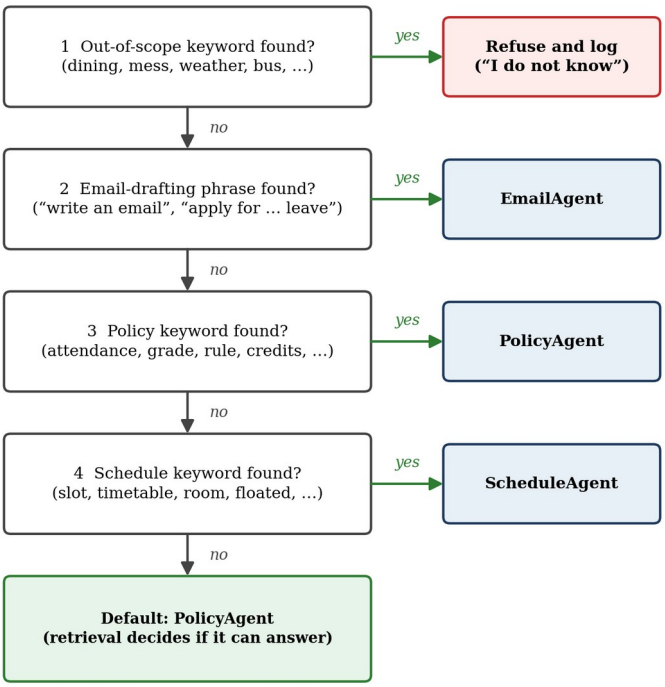


Figure 2: *Priority-ordered intent routing in Orchestrator.classify_intent().*

3.4 Project layout

Path	Contents
src/app.py, src/config.py	CLI entry point; paths and settings loaded from .env via python-dotenv
src/agents/	orchestrator.py, policy_agent.py, schedule_agent.py, email_agent.py
src/tools/	Six tool modules plus package exports
src/rag/retriever.py	AcademicRetriever – ChromaDB query, threshold filter, citation formatting
scripts/ingest_documents.py	Offline PDF → chunk → embed → index pipeline
data/	documents/ (PDF and CSV sources), four structured CSV files, chromadb/ persistent index
evaluation/	test_prompts.json, evaluate.py, results/evaluation_report.md
logs/unanswered_questions.jsonl	Append-only log of refused queries

4 Technologies and Libraries Used

The table separates what the executed code actually imports from what appears only in the plan or in requirements.txt. This distinction matters for interpreting the results.

Technology	Status in code	Role
Python 3.13	Used	Implementation language (bytecode in __pycache__ is cpython-313)
ChromaDB (PersistentClient)	Used	Local vector store; one collection academic_docs, cosine HNSW index, SQLite metadata
Chroma default embedding function	Used	Local sentence-embedding model bundled with ChromaDB; applied at both index and query time
PyMuPDF (fitz)	Used (ingestion)	Page-by-page text extraction from PDFs
Pandas	Used	Loading and filtering the four CSV tables

Technology	Status in code	Role
python-dotenv	Used	Reads .env (model names, Chroma directory, log path)
re, json, glob, pathlib	Used	Query parsing (course codes, durations), logging, file handling
Google ADK	Listed only	In requirements.txt and the plan; not imported by any source file
LiteLLM / Ollama (Qwen3-8B, nomic-embed-text)	Listed only	Configured in .env / config.py; not called at runtime
python-docx, requests, colorama	Listed only	Present in requirements.txt; unused in the delivered modules

Two points follow. First, the embedding model is the ChromaDB default rather than nomic-embed-text, so no Ollama service is needed to run retrieval. Second, because no generative model is involved, answer wording is produced by Python string templates and by returning retrieved text, which makes behaviour reproducible but limits flexibility (Section 8).

5 Detailed Implementation of the Main Modules

5.1 Document ingestion (scripts/ingest_documents.py)

The ingestion script globs every *.pdf in data/documents/ (33 files are present; 32 are indexed – see Section 7.1), extracts text with PyMuPDF one page at a time, normalises whitespace with clean_text(), and splits each page with chunk_text(). Chunking is word-based with a window of 600 words and an overlap of 100 words; fragments shorter than 20 characters are dropped. Each chunk is stored with the metadata source, page, title and chunk_index, and a deterministic ID of the form <file>_p<page>_c<index>. Documents are added to ChromaDB in batches of 200, and the existing collection is deleted and rebuilt on every run so that the index never contains stale chunks.

```
def chunk_text(text, chunk_size=CHUNK_SIZE, overlap=CHUNK_OVERLAP):
  words = text.split(); chunks = []; i = 0
  while i < len(words):
    chunk = " ".join(words[i:i + chunk_size])
    if len(chunk.strip()) > 20: chunks.append(chunk)
    i += chunk_size - overlap
  return chunks
```

Two circulars (amendment_attendance_policy_2024.pdf, amendment_examination_policy_2024.pdf) are scanned images with no text layer. The script therefore contains a dictionary SCANNED_PDF_TRANSCRIPTS holding a manual transcription of each page, which is chunked and indexed in place of PyMuPDF output. This is how the key attendance and examination amendments become searchable.

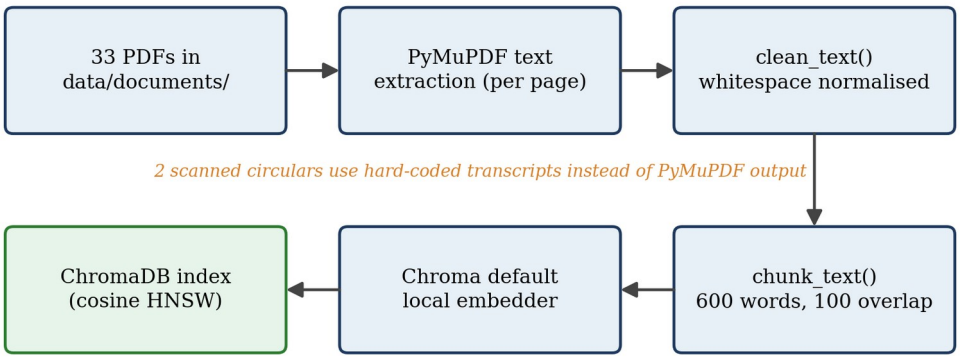


Figure 3: *Offline ingestion pipeline (scripts/ingest_documents.py).*

5.2 Retrieval (src/rag/retriever.py, src/tools/rag_search.py)

AcademicRetriever opens the persistent client, obtains the academic_docs collection and attaches the same default embedding function used at ingestion. query() asks ChromaDB for the top_k nearest chunks, returns documents, metadata and distances, and discards any match whose distance exceeds a threshold of 1.2. format_citation() de-duplicates (source, page) pairs and renders the block that is appended to answers:

> Sources:
> - academic_regulations_iitj_2019.pdf (Page 7)
> - amendment_attendance_policy_2024.pdf (Page 1)

search_knowledge_base() is the thin tool wrapper used by the PolicyAgent. It keeps a module-level singleton retriever so the collection is opened once per process.

5.3 Orchestrator (src/agents/orchestrator.py)

The orchestrator is the only component that sees every query. Its classify_intent() method applies four rule groups in strict priority order:

- **Out-of-scope:** any of 25 keywords such as dining, mess, canteen, taxi, bus, weather, movie, hotel.
- **Email:** phrases such as “write an email”, “draft an email”, or “apply for” together with “leave”.
- **Policy:** terms such as attendance, grade, rule, regulation, policy, “credits do i need”, degree, “make-up”.
- **Schedule:** slot, timetable, room, “when is”, floated, or elective combined with a counting word.

Anything unmatched defaults to the PolicyAgent, so that the retrieval layer – not a keyword list – decides whether the knowledge base can answer. In handle(), an out-of-scope result produces the fixed message “I do not know. I can only assist with academic queries…”, calls log_unanswered_question() and returns refused=True, logged=True; other intents are delegated and the intent label is attached to the response.

5.4 PolicyAgent (src/agents/policy_agent.py)

The PolicyAgent first calls search_knowledge_base(query, top_k=4). If nothing passes the distance filter it replies “I do not know. The requested information was not found in the official academic documents.” Otherwise it builds an answer in one of four ways: three **curated templates** for the most common questions (attendance and the Major exam; Open Elective credits for B.Tech.; make-up examinations) and a **generic path** that returns the text of the best chunk plus the second-best chunk as “additional context”. The attendance template, for instance, reflects the 2024 amendment: attendance should not fall below 75 %, examination performance is no longer linked to attendance, and persistent absence can lead to de-registration or semester withdrawal. In every case the citation block produced by the retriever is appended.

The curated templates were checked against the indexed text for this report (Section 7.3): the 12-credit Open Elective requirement appears in the CSE, ECE, EE, AI&DS, Civil and other B.Tech. curriculum chunks (the template’s “outside the home department” wording is a broad paraphrase of the curricula’s “interdisciplinary courses beyond the programme”), and the make-up rule (application within ten days, through the course instructor to the Dean (Academics)) appears in the 2019 regulations.

5.5 ScheduleAgent (src/agents/schedule_agent.py)

The ScheduleAgent has two branches. The **slot branch** is triggered by words like slot, timing, room or “when does”. It extracts a course code with the regular expression \b([A-Za-z]{2,3}\d{4})\b, or matches the query against a list of 20 known course titles, and calls lookup_timetable(). The first matching row is formatted as slot, days, time, room and instructor. The **catalogue branch** is triggered by electives, floated, department or courses. It maps department words to codes (Humanities/HSS → HSS, Electrical → EE, Computer Science → CSE, Mechanical → ME), scans for a level in {100,…,700}, and calls lookup_courses(). The response reports the count and lists each course with credits, type and instructor. If neither branch applies, a free-text search over title, code and description is attempted.

The tools use lazily-loaded module-level DataFrames. lookup_timetable() filters by course code, course title or slot letter; lookup_courses() filters by department (with an HSS/Humanities alias set), numeric level, course type and keyword.

5.6 EmailAgent and email tools

The EmailAgent extracts the course code with the same regular expression (defaulting to CSL7570 if none is found), the number of days with (\d+)\s*days? (default 2), and decides the email type: “makeup”, “make-up” or “missed exam” yields makeup_exam, everything else sick_leave. draft_email() calls get_instructor_email(), which looks up the course in instructor_course_mapping.csv (falling back to a faculty-name search in faculty.csv) and returns the instructor’s name and address. A formal template is then filled with subject, salutation, reason, commitment to catch up, and signature block. The response always ends with the note that the draft **has not been sent**.

5.7 Unanswered-question logger and CLI

log_unanswered_question() appends a JSON object {timestamp (UTC ISO-8601), question, reason} to logs/unanswered_questions.jsonl, creating the directory if necessary. The CLI in src/app.py forces UTF-8 output for Windows terminals, loops on input(), handles exit/quit/Ctrl-C, and prints [Agent | Intent] above each answer.

6 Algorithms and Design Decisions

6.1 Retrieval-augmented answering

Retrieval is dense nearest-neighbour search: the query and each chunk are embedded into the same vector space and ChromaDB’s HNSW index returns the closest chunks under cosine distance. Because the collection was created with hnsw:space = cosine, distances lie in [0, 2], so a threshold of 1.2 removes only clearly dissimilar chunks; it is a permissive filter rather than a strict relevance gate. The consequence is that an unrelated academic-sounding query will usually still return some chunk, and refusal of non-academic queries is therefore handled earlier, by keyword rules in the orchestrator.

6.2 Chunking

A 600-word window with 100-word overlap was chosen so that a rule and its qualifying sentence are likely to fall in the same chunk. In practice (Figure 5) most pages are shorter than the window, so the typical chunk is a single page or half-page (mean 263 words, median 279); only two chunks reach the 600-word cap. Page-level splitting has a useful side-effect: the page metadata is exact, which is what makes page-accurate citation possible.

6.3 Why rules rather than a language model for routing

Intent routing uses ordered keyword rules. The advantages are determinism, zero latency, no model memory footprint and easy unit testing – the same query always reaches the same agent. The disadvantages are brittleness to paraphrase and to substring collisions (Section 8). The ordering encodes a safety decision: refusal is checked first, so a non-academic request can never be sent to retrieval.

6.4 Structured data versus vector search

Questions that need exact answers – a slot letter, a room, a count of 400-level HSS electives – are answered from CSV tables with Pandas filters and never from embeddings. Counting and exact lookup are precisely where approximate retrieval is least reliable, so the design routes them to deterministic code. Policy text, which is long and unstructured, goes to the vector store.

6.5 Grounding and citation

Grounding is enforced structurally: the PolicyAgent can only respond with templates verified against the corpus or with retrieved text, and it appends the page-level citation produced by the retriever. When retrieval returns nothing, the reply is “I do not know”. Email drafts are explicitly labelled as unsent, keeping the human in the loop.

6.6 Other decisions

- **Full re-index on each ingestion run** – simple and guarantees consistency between the PDFs and the index.
- **Manual transcripts for scanned circulars** – keeps authoritative amendments searchable without an OCR dependency.
- **Dummy structured data** – the faculty, timetable and course CSVs use illustrative names and @iitj.ac.in addresses, consistent with the plan’s instruction to avoid real personal data.

- **Tools as plain Python functions** – each has a documented signature, so they could be wrapped as ADK FunctionTools without change.

6.7 Plan versus delivered implementation

Aspect	Planned (assignment_plan.md)	Delivered code
Agent framework	Google ADK with LLM sub-agents and transfer	Plain Python classes, rule-based routing
Generation	Qwen3-8B / Llama 3.1-8B via LiteLLM + Ollama	No generative model; templates and retrieved text
Embeddings	Ollama nomic-embed-text	ChromaDB default local embedder
Chunker	LangChain recursive splitter, 512 / 64	Word window 600 / 100
Documents	20–30 documents	32 indexed PDFs (+ CSV sources not indexed)
Evaluation	Two models × 3 runs, consistency analysis	One deterministic system, 20 prompts, single run

7 Results, Testing and Diagrams

7.1 Knowledge base statistics

Inspection of the persisted ChromaDB database gives the following facts: one collection (academic_docs), **1,075 chunks** from **32 distinct PDFs**, a SQLite file of about 25.6 MB, and chunk lengths between 2 and 600 words. Curricula dominate the index, led by the Civil and Infrastructure booklet (271 chunks), followed by Mechanical (70), Aerospace (65) and Materials (64); the 2019 Academic Regulations contribute 50 chunks and the PG regulations 38 (Figure 4). The documents folder holds 33 PDFs, but the October non-vegetarian mess menu PDF is not among the indexed sources, and the three CSV files in that folder (mess menu, an SME course list, timetable-committee members) are not ingested; the data folder also contains a stale second HNSW segment directory left from an earlier ingestion run.

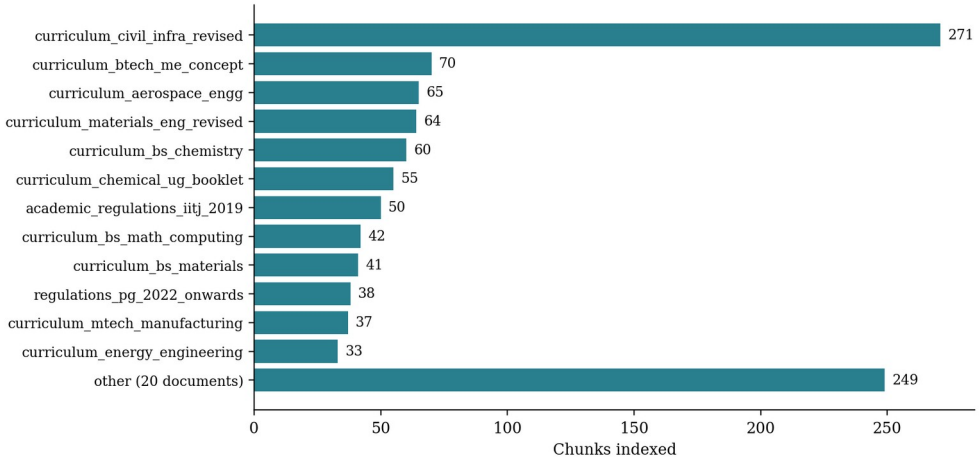


Figure 4: *Chunks per source document (1,075 chunks from 32 PDFs).*

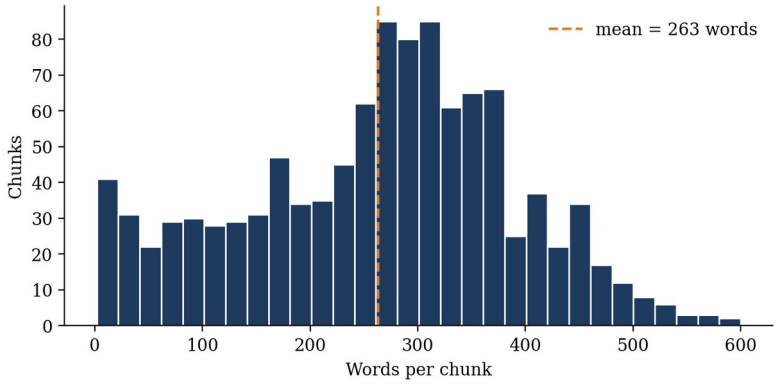


Figure 5: *Distribution of chunk lengths (cap = 600 words).*

7.2 Test suite and recorded results

evaluation/test_prompts.json defines 20 prompts. Each carries an expected agent, expected keywords, a flag for whether a citation is required and a flag for whether refusal is expected. evaluate.py runs every prompt through the orchestrator and marks a test as passed if (i) the answering agent equals the expected agent, (ii) at least one expected keyword appears in the answer, (iii) refusal behaviour matches, and (iv) a "Sources"/.pdf citation is present when required. The recorded report in evaluation/results/evaluation_report.md states **20/20 passed (100 %)** and **20/20 routing accuracy**.

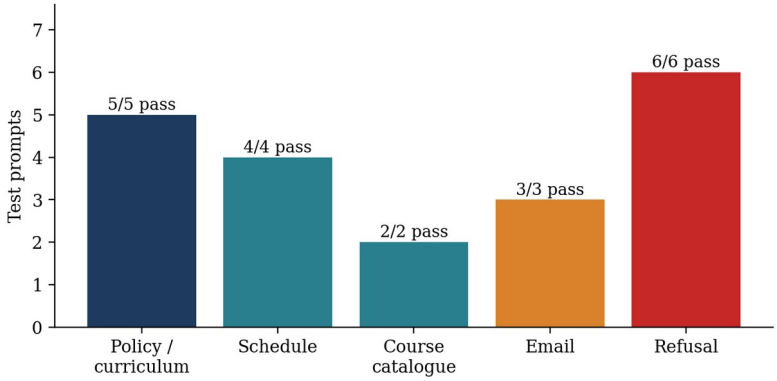


Figure 6: *Recorded results of the 20-prompt suite by category.*

ID	Category	Prompt (abridged)	Agent	Result
T01	Policy	Attendance requirement for Major exam	Policy	Pass
T02	Schedule	Slot for Autonomous Systems	Schedule	Pass
T03	Catalogue	400-level HSS electives floated	Schedule	Pass
T04	Curriculum	Open Elective credits for B.Tech.	Policy	Pass
T05	Email	Leave email for CSL7570, 2 days	Email	Pass
T06	Refusal	Campus dining options	Orchestrator	Pass
T07	Refusal	Reaching campus from city centre	Orchestrator	Pass
T08	Schedule	Slot and room, Operating Systems	Schedule	Pass
T09	Schedule	When Machine Learning meets	Schedule	Pass
T10	Policy	Make-up exam policy (illness)	Policy	Pass
T11	Email	Make-up email for CSL3060	Email	Pass
T12	Refusal	Weather forecast for Jodhpur	Orchestrator	Pass
T13	Refusal	Food delivery to hostel	Orchestrator	Pass

ID	Category	Prompt (abridged)	Agent	Result
T14	Catalogue	Courses floated by EE	Schedule	Pass
T15	Policy	Absence > 2 weeks without leave	Policy	Pass
T16	Policy	Major exam duration and weightage	Policy	Pass
T17	Schedule	Slot and timing, MEL3010	Schedule	Pass
T18	Email	3-day leave email, CSL4010	Email	Pass
T19	Refusal	Movie tickets near campus	Orchestrator	Pass
T20	Refusal	Bus / cab to railway station	Orchestrator	Pass

7.3 Sample interactions

The following transcripts were produced by running the project's own agent classes on the actual CSV data. The vector store could not be loaded in the environment used to prepare this report (ChromaDB was not installable), so retrieval was stubbed; the schedule, email and refusal paths – which do not use retrieval – therefore ran exactly as in the submission.

```
User > What is the slot for the course Autonomous Systems?
[Agent: ScheduleAgent | Intent: schedule]
The course Autonomous Systems (CSL7570) is scheduled in Slot D.
- Days: Mon/Wed/Fri   - Time: 11:00-11:50
- Room: LH-101        - Instructor: Dr. Rahul Sharma (CSE Dept)

User > How many 400 level electives have been floated by the Humanities
       department in this semester?
[Agent: ScheduleAgent | Intent: schedule]
There are 3 400-level courses/electives floated by HSS department:
- HSL4010: Philosophy of Mind (3 credits, OE, Dr. Rajesh Pandey)
- HSL4020: Ethics and Technology (3 credits, OE, Dr. Sunita Jain)
- HSL4030: Economics of Innovation (3 credits, OE, Dr. Arun Mishra)

User > What are the available dining options on the campus?
[Agent: Orchestrator | Intent: out_of_scope]
I do not know. I can only assist with academic queries such as course
curriculum, rules & regulations, timetable, and drafting emails.
```

```
User > Write an email to the course instructor of CSL7570 that I need leave for 2 days
[Agent: EmailAgent | Intent: email]
To: rahul.sharma@iitj.ac.in (Dr. Rahul Sharma)
Subject: Application for Sick Leave: CSL7570 (Autonomous Systems) - B21CS001

Respected Dr. Rahul Sharma,
I am writing to inform you that I am unable to attend classes for CSL7570:
Autonomous Systems for 2 days due to severe viral illness and medical rest
advised by physician. I will ensure that I catch up on all course materials ...
Sincerely, Antigravity Student, Roll No: B21CS001, Dept. of CSE, IIT Jodhpur

Note: This email draft has been generated for your review. It has NOT been sent.
```

Verification of curated policy answers. Because the PolicyAgent's templates are fixed text, their factual content was checked against the chunks stored in the index. (a) The 75 % attendance wording and the C-grade rule of the 2019 regulations appear on page 7 of academic_regulations_iitj_2019.pdf, and the 2024 circular (transcribed on pages 1–2) supersedes it – the template follows the circular. (b) A 12-credit Open Elective requirement is stated in the CSE, ECE, EE, AI&DS and Civil curriculum chunks, e.g. "Students in the regular B.Tech programme are required

to complete 12 credits of open elective courses." (c) The make-up rule – application within ten days of the missed examination – is in section 8 of the 2019 regulations (page 11) and the PG regulations (page 13). The templates are consistent with the corpus.

7.4 Unanswered-question log

logs/unanswered_questions.jsonl contains 19 entries covering 6 distinct refused questions (dining ×6, reaching campus ×5, and weather, food delivery, movie tickets and bus/cab ×2 each). The repeated entries come from running the quick test and the evaluation several times; the logger does not de-duplicate, which is acceptable for an audit trail but would need aggregation for a human reviewer.

7.5 Critical reading of the results

The 100 % pass rate should be interpreted with care. (1) The suite was written by the same author as the system, and its keywords mirror the curated templates, so it measures regression rather than generalisation. (2) Keyword matching accepts an answer if **any one** expected keyword is present. (3) Refusal tests have requires_citation = false, yet the stored report prints "Yes" in the Citations column for them, because the script sets has_citation = True whenever no citation is required – the column is therefore not evidence of grounding for those rows. (4) The schedule and email prompts use course codes and titles that exist in the dummy CSVs, so they are lookups by construction. (5) The consistency and cross-model comparison planned in the assignment are trivially satisfied for a deterministic system and were not run. (6) For this report the retrieval-dependent paths were verified by inspecting stored chunks rather than by re-running vector search. Within these limits the results do show that routing, tool wiring, the refusal path and logging work end to end.

8 Limitations and Possible Improvements

Observed limitation	Proposed improvement
No language model in the loop. Policy answers are either curated templates or raw chunk text; paraphrased or compound questions get verbatim passages.	Add an LLM (the planned Qwen3-8B / Llama 3.1-8B through LiteLLM and Google ADK) that answers *only* from retrieved context and must quote the citation; keep templates as a fallback.
Keyword routing causes substring false positives. Tested with the project's classifier: "Business Economics course" and "training requirement" are refused as out-of-scope because bus and train occur inside the words; mess (in "message") and cab (in "vocabulary") behave the same way, and in the ScheduleAgent the department test me matches inside many ordinary words.	Match whole words with regular expressions (\bbus\b) or tokenise; better, use an LLM or embedding-based intent classifier with a confidence threshold.
Out-of-scope refusal is a blacklist. Questions such as "capital of France" are not caught by the orchestrator and depend on retrieval returning nothing; with a 1.2 cosine threshold, some chunk will usually pass.	Tighten the distance threshold, require a minimum relevance score, and add a whitelist/embedding-similarity check to the academic domain.
Slot argument unused. "Which courses are offered in Slot D?" is routed to the schedule agent, but lookup_timetable(slot=…) is never called, so the first table row is returned.	Parse slot letters and return all matching courses; support day/time-based queries.
Email agent hard-codes details. Student name ("Antigravity Student"), roll number and department are fixed; the reason is chosen by email type rather than taken from the user; a missing course code silently defaults to CSL7570.	Collect or configure student identity, extract the stated reason and dates, ask for the course code when absent, and let the user choose a tone.
Static dummy structured data. Timetable, faculty and floated-course CSVs are illustrative and not synchronised with the official 2026–27 slot system CSV shipped in data/documents/.	Ingest the official course/slot spreadsheets into the structured layer and validate CSVs at load time.

Observed limitation	Proposed improvement
Corpus gaps. The mess menu and three CSVs in data/documents/ are not indexed; scanned PDFs need manual transcripts; the index contains many near-duplicate calendars and curricula with no recency or programme filter.	Add OCR, index CSV/Excel sources, attach programme and year metadata, and filter or re-rank by it.
Evaluation is narrow. 20 self-authored prompts, keyword matching, single run.	Build a held-out set with reference answers, score faithfulness and citation correctness, include paraphrases and adversarial prompts, and compare models over repeated runs.
Interface and operations. CLI only; the unanswered log has no de-duplication; stale HNSW directory remains.	Add a web UI, aggregate the log by question, and clean unreferenced index segments.

9 Conclusion

The assignment delivers a working, modular Academic Helpdesk for IIT Jodhpur. An orchestrator routes each query to a policy, schedule or email agent; policy answers draw on a 1,075-chunk ChromaDB index of 32 official PDFs and carry page-level citations; schedule answers come from exact CSV lookups; emails are drafted for the correct instructor and never sent; and out-of-scope questions are refused and logged. On the recorded 20-prompt suite all tests pass and routing is perfect, and the curated policy answers were confirmed against the indexed source text.

The most important caveat is architectural: the delivered system is rule-based and does not yet use the Google ADK / Ollama language-model layer described in the plan, so its strengths – determinism, auditability, low resource use – come with limited language understanding. The same modularity that makes the current system easy to test makes the upgrade path clear: replace the keyword classifier and answer templates with LLM-driven agents that call the existing tools, retain the retriever and the refusal-and-logging policy as safety layers, and extend the evaluation to a held-out, multi-model benchmark. With those changes the helpdesk could move from a reliable demonstrator to a genuinely flexible assistant for students.

Appendix – Reproducing the system

```
pip install -r requirements.txt
python scripts/ingest_documents.py     # rebuilds data/chromadb (32 PDFs -> 1,075 chunks)
python src/app.py                      # interactive helpdesk CLI
python test_quick.py                   # seven assignment queries
python evaluation/evaluate.py          # 20-prompt suite -> evaluation/results/evaluation_report.md
```